Politecnico di Milano

Ingegneria del Software – a.a. 2004/05

Appello dell'8 febbraio 2006

Cognome

Nome

Matricola

Sezione (segnarne una) Baresi ☐, Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 26/30.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 10)

Il metodo statico *controlla()* riceve in ingresso un array *nums* e un intero *n* positivo e restituisce un valore booleano. L'array nums, di lunghezza non precisata, contiene dei numeri interi, per ipotesi tutti diversi tra loro. Il metodo restituisce true se ognuno dei primi n numeri dell'array è più piccolo di ognuno dei numeri che occupano le posizioni da n alla dimensione dell'array. L'array non viene modificato.

b - Si fornisca la specifica JML del metodo. Si rammenta che la specifica deve considerare i casi in cui l'array sia nullo o di lunghezza minore di n e quello in cui gli elementi non siano tutti distinti fra loro.

Soluzione

```
//@assignable nothing;
//@ requires nums!=null && n<nums.length &&
                (\forall int i; 0<=i< nums.length;
                        (\forall int j; i<j<nums.length; nums[i]|!=nums[j]));
//@ ensures \result ==
                        (\forall int i; 0<=i<n;
                                (\forall int j; n<=j<nums.length; nums[i]<nums[j]));
```

public static boolean controlla(int [] nums, int n)

b - Si trasformi la specifica JML del punto precedente sostituendo una a scelta delle precondizioni nel lancio di opportune eccezioni.

Soluzione

Data una opportuna classe eccezione IndiceFuoriDaiLimitiException:

```
//@assignable nothing;
//@ requires nums!=null&&
                (\forall int i; 0<=i< nums.length;
                        (\forall int j; i<j<nums.length; nums[i]|!=nums[j]));
//@ ensures \result ==
                        (\forall int i; 0<=i<n;
                                (\forall int j; n<=j<nums.length; nums[i]<nums[j]));
//@signals (IndiceFuoriDaiLimitiException e) n>=nums.length;
public static boolean controlla(int [] nums, int n) throws IndiceFuoriDaiLimitiException
```

Esercizio 2 (punti 9)

Si consideri la classe ***InsPunti***, classe immutabile che rappresenta un insieme di punti (senza duplicazioni) in un piano. Ogni punto dell'insieme è univocamente individuato da un indice intero, compreso fra 0 e la cardinalità dell'insieme (meno uno). La classe fornisce, oltre a costruttori e produttori che non sono qui riportati, le primitive seguenti:

```
class InsPunti  {
......
//restituisce la cardinalità dell'insieme
int numPunti() {}

//restituisce la distanza tra due punti dell'insieme, individuati dai loro indici i e j,
//con 0 ≤ i,j < numPunti()
float distanza(int i, int j){}

//restituisce il diametro dell'insieme, cioè la distanza massima tra coppie di punti dell'insieme
float diametro(){}
......
}
```

a – Si fornisca la specifica JML (pre- e post-condizione) del metodo diametro()

Soluzione

```
//@assignable \nothing
//@ requires this.numPunti() >= 2;
//@ ensures (\exists int i; 0<=i<this.numPunti();
                    (\exists int j; 0<j<this.numPunti();
                            \result == this.distanza(i,j) &&
                            (\forall int x; 0<=x<this.numPunti();
                            (\forall int y; 0<y<this.numPunti(); \result >= this.distanza(x, y)
            )) ))
float diametro(){}
```

b –Il REP per la classe viene scelto come segue:

Un array di float `coord`, con 2·n componenti (l'insieme contiene n punti), in cui viene messa la rappresentazione polare di ogni punto (modulo, angolo). I primi due elementi di `coord` memorizzano rispettivamente modulo e angolo del primo punto, il terzo e il quarto modulo e angolo del secondo punto, etc.

Scrivere in JML l'invariante di rappresentazione, considerando che l'insieme può essere vuoto ma non può contenere punti duplicati.

Soluzione

```
//@ private invariant coord != null &&
//@ (\exists int n ; n>=0 ; 2*n == coord.length &&    coord ha un numero pari di elementi
//@     (\forall int i; 0<=i<n; coord[2*i]>=0) &&          condizione sul modulo di tutti i punti
//@     (\forall int i; 0<=i<n; 0<coord[2*i+1]<=6.28 )&&       condizione sull'angolo di tutti i
punti
//@     (\forall int x; 0<=x<n;                                    tutti i punti diversi tra di loro
//@             (\forall int y; 0<=y<n;
//@                     x !=y ==> coord[2*x] != coord[2*y] || coord[2*x+1] != coord[2*y+1] ) ));
```

c – Si fornisca la specifica JML del metodo boolean triangolo(int i, int j, int k){} per sapere se i tre punti, identificati dai loro indici e ipotizzati essere fra loro distinti, possono essere i vertici di un triangolo non degenere. A tal fine si ipotizzi che un triangolo sia non degenere se e solo se la lunghezza di ognuno dei suoi lati è minore della somma della lunghezza degli altri due lati.

Soluzione

Per verificare che i tre punti distinti non siano allineati, basta imporre le tre disuguaglianze triangolari.

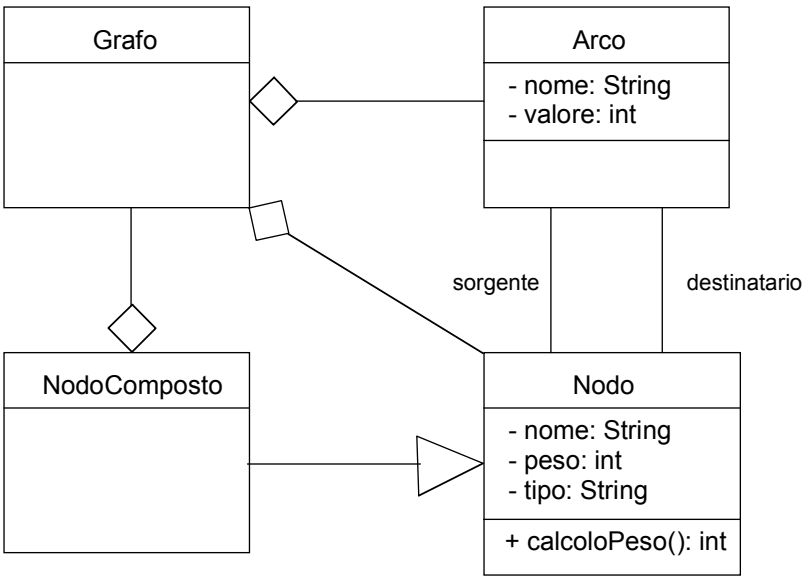
```
//@assignable \nothing
//@ requires this.numPunti() >= 3 && 0<=i,j,k <numPunti() && i != j && j != k && k != i;
//@ ensures \result == (distanza(i,j) < distanza(i, k) + distanza(k, j) &&
                distanza(j,k) < distanza(j, i) + distanza(i, k) &&
                distanza(k,i) < distanza(k, j) + distanza(j, i) );
boolean triangolo(int i, int j, int k){}
```

Esercizio 3 (punti 8)

Disegnare un diagramma delle classi che rappresenti un *grafo gerarchico*.

Un grafo gerarchico è un insieme di nodi e archi opportunamente connessi, in cui ogni arco connette esattamente due nodi, e ogni nodo può essere "semplice" o "composto". Un nodo semplice non è ulteriormente decomposto, mentre un nodo composto contiene al suo interno ancora un grafo gerarchico.
Pertanto, la relazione di contenimento è iterata gerarchicamente: un nodo composto, parte di un grafo, contiene un altro grafo, i cui nodi composti contengono altri grafi e così via.

Ogni *arco* è orientato e collega un nodo, detto *sorgente* dell'arco, a un nodo detto *destinazione*. Un arco è caratterizzato inoltre da un nome e un colore.

Ogni nodo è identificato da un nome (univoco al suo livello gerarchico) e da un peso. Se il nodo è composto, il suo peso è dato dalla somma dei pesi dei nodi in esso contenuti, ad ogni livello. I nodi semplici si dividono ulteriormente in: nodi pozzo, nodi sorgente e nodi normali. I primi possono avere solo archi entranti, ma non uscenti; i secondi possono avere solo archi uscenti, ma non entranti, e i terzi non pongono restrizioni.

Soluzione (per brevita', non sono riportate le cardinalita'). E' corretto, ma un po' macchinoso, anche definire tre sottoclassi di Nodo (sorgente, pozzo, normale) per rappresentare i vari tipi di nodo, a patto poi di indicare le associazioni corrette con la classe Arco.